

Appunti del 12/03/2026 — Partizionamento e Raggruppamenti

1. Partizionamento e Raggruppamento

Il partizionamento (o raggruppamento) è una tecnica che si applica quando si ha una relazione 1 a molti tra entità, come nel caso di un Blog che contiene Blog Post.

Nell'esempio, un singolo Blog può avere fino a un massimo di 20 post ($M = 20$). Invece di trattare tutti questi post come una lista piatta e indistinta, li si organizza in gruppi logici secondo un criterio — in questo caso il mese di pubblicazione.

Così, il Java Blog presenta i post suddivisi: 3 post a marzo 2026, 3 a febbraio 2026, 1 a gennaio 2026, 1 a dicembre 2023. Ogni partizione rappresenta un sottoinsieme coerente e delimitato dei dati.

Vantaggi del partizionamento

- **Query più efficienti:** invece di scorrere l'intera collezione si accede direttamente alla partizione di interesse.
- **Dati organizzati e prevedibili:** ogni gruppo ha una dimensione naturalmente limitata.
- **Gestione semplificata:** archiviazione o eliminazione di dati vecchi partizione per partizione anziché record per record.

2. Lista piatta vs Lista di Liste

Senza partizionamento, il Blog1 (istanza di tipo Blog) contiene un campo posts di tipo `List<BlogPost>` che punta direttamente a tutti i post. Man mano che crescono, questa lista diventa indefinitamente lunga — problematico nei database a documenti come MongoDB, che ha un limite di 16MB per documento.

Con il partizionamento si passa concettualmente da una lista piatta:

`Blog → [P1, P2, P3, P4, P5, ...]`

a una lista di liste:

`Blog → [ [P1] , [P2, P3, P4] , [P5, P6] ]` (Gennaio /
Febbraio / Marzo)

3. Criteri di Partizionamento

La scelta del criterio non è neutrale: dipende da come si accede ai dati. Dai post di esempio (P1–P6, con categorie Java/SQL/JS/TS e date dal gennaio al aprile 2026):

- **Per mese:** 3 gruppi — Gennaio (P1), Febbraio (P2, P3, P4), Marzo-Aprile (P5, P6).
- **Per categoria:** 4 gruppi — Java, SQL, JS, TS.

4. Partizionamento Gerarchico (Anno → Mese → Post)

Quando il volume è molto grande si introduce un livello in più: prima si raggruppa per anno, poi per mese, poi si trovano i post. Si ottiene una lista di liste di liste:

```
Blog → [ 2026 → [ Gennaio → [P1, P2] ], 2025 → [...], ... ]
```

Invece di scorrere tutti i post in sequenza, si naviga la gerarchia scendendo di livello in livello — prima l'anno, poi il mese — lavorando ogni volta su un insieme molto più piccolo di dati.

5. Implementazione Java

Classe PostsByYear

Si introduce una classe dedicata `PostsByMonth`, e si crea un array di 12 elementi di quel tipo — aggirando elegantemente il problema.

```
PostsByMonth[] months = new PostsByMonth[12];
```

Il costruttore di `PostsByYear` riceve l'anno, lo salva, e inizializza tutti e 12 i mesi con un ciclo `for` passando il numero del mese (`i+1`, da 1 a 12) e l'anno.

Metodo getPostsByYearAndMonth()

Il metodo restituisce una `Map<Integer, PostsByYear>` e fa tre cose in sequenza:

- **Prima:** scorre tutti i post per trovare l'anno minimo e massimo (usando valori estremi iniziali: `minYear = 4500`, `maxYear = 0`).
- **Poi:** crea una `LinkedHashMap` e la popola con un `PostsByYear` vuoto per ogni anno nell'intervallo trovato.
- **Infine:** scorre di nuovo tutti i post e per ciascuno naviga nella mappa fino alla posizione giusta con `res.get(year).months[month].posts.add(p)`.
Nota: il mese usa -1 perché l'array è 0-indexed.

Al frontend deve arrivare la mappa che esce da `getPostsByYearAndMonth()`.

6. Ripasso MySQL

MySQL è un sistema di gestione di database relazionale (RDBMS) basato su SQL. I dati sono organizzati in tabelle (righe e colonne), contenute in un database.

Operazioni CRUD

INSERT (inserire), **SELECT** (leggere), **UPDATE** (modificare), **DELETE** (eliminare).

Chiavi

- **PRIMARY KEY:** identifica univocamente ogni riga.
- **FOREIGN KEY:** collega due tabelle referenziando la chiave primaria di un'altra.

ON DELETE RESTRICT / ON UPDATE CASCADE

- **ON DELETE RESTRICT:** blocca la cancellazione del record padre se esistono figli che lo referenziano.
- **ON UPDATE CASCADE:** propaga automaticamente la modifica della chiave primaria del padre a tutti i figli.

Query SQL — Conteggio post per anno e mese

```
SELECT year(published_on) as year, month(published_on) as
month, count(*)
FROM voices.blog_post
WHERE blog_id = 1
GROUP BY year(published_on), month(published_on);
```

Seleziona anno, mese e numero di post dalla tabella `blog_post`, filtrando per blog con `ID = 1` e raggruppando per anno e mese. `count(*)` conta tutte le righe in ciascun gruppo.